

Greedy Algorithms

Review Notes

A greedy algorithm makes the locally best choice that can be proved safe, then never undoes it. Unlike backtracking, it does not enumerate paths or maintain a `visited` set. The real difficulty is proving why today's choice cannot damage the future.

- Python 3
- tutor style
- visual explanations
- FAQ included
- 7 core patterns

00 Review Order + Three Universal Questions

① What local state matters most?
Identify the state.

② Can this choice harm the future?
Prove the choice is safe.

③ Must I explore several routes?
Decide whether branching is needed.

01 · START

Intervals

sort by end

02

Coin Change

not always optimal

03

Cookies

two pointers

04

Jump Game

farthest

05

Min Arrows

interval coverage

06

Partition Labels

last occurrence

07

Stock Profit

lowest price + best profit

Question ③ is the key test. If the answer is “no,” avoid recursion, `path/pop`, and `visited`. The usual greedy skeleton is optional sorting + one pass + a few state variables.

01 When Should You Think Greedy?

Look for maximum/minimum goals, many non-conflicting choices, matching sorted arrays, reachability, or a state such as the farthest, earliest, lowest, or highest value.

Method	Main idea
Backtracking	Try several possibilities and undo failed choices.
Dynamic Programming	Store and compare optimal answers to subproblems.
Greedy	Keep the best safe local state and move forward without retreating.

02 Maximum Non-overlapping Intervals

MUST KNOW

Select the maximum number of compatible intervals by always choosing the interval that finishes earliest.



max_non_overlapping_intervals · sort by end time

greedy · sort

```
def max_non_overlapping_intervals(intervals):
    if not intervals:
        return 0

    sorted_intervals = sorted(
        intervals,
        key=lambda interval: interval[1] # sort by end time
    )
    count = 0
    last_end = float("-inf")

    for start, end in sorted_intervals:
        if start >= last_end: # touching is allowed
            count += 1
            last_end = end
    return count
```

Why end time? An interval that starts early can still run for a long time and block several shorter choices. Finishing early is what preserves future capacity.

Why `start >= last_end` instead of `>`?

Use `>=` when touching endpoints are compatible. Confirm the problem's boundary rule before coding.

03 Greedy Coin Change

CAUTION: NOT ALWAYS OPTIMAL

Repeatedly take the largest coin no greater than the remaining amount.

greedy_coin_change

largest first

```
def greedy_coin_change(coins, amount):
    result = []
    sorted_coins = sorted(coins, reverse=True)

    for coin in sorted_coins:
        while coin <= amount:
            amount -= coin
            result.append(coin)
    return result
```

Works here

`[1,5,10,25]`, amount 41 → `[25,10,5,1]`

Fails here

`[1,3,4]`, amount 6 → greedy `[4,1,1]`, optimum `[3,3]`

Largest-first is a rule, not a theorem. For the general minimum-coin problem, use DP because it compares alternative coin choices instead of committing permanently.

Why is `coins(len(coins)-1-i)` wrong?

Parentheses call a function, but `coins` is a list. Use square brackets for indexing, or simply iterate through `sorted_coins`.

04 Assign Cookies

Give the smallest suitable cookie to the least demanding child. Never waste a large cookie when a smaller one is sufficient.

find_content_children · sorting + two pointers

two pointers

```
def find_content_children(greed, cookies):
    sorted_greed = sorted(greed)
    sorted_cookies = sorted(cookies)
    child_index = 0
    cookie_index = 0

    while (child_index < len(sorted_greed)
           and cookie_index < len(sorted_cookies)):
        if sorted_cookies[cookie_index] >= sorted_greed[child_index]:
            child_index += 1      # child satisfied
            cookie_index += 1     # cookie consumed
    return child_index
```

Why advance a cookie that is too small?

If it cannot satisfy the least demanding remaining child, it cannot satisfy any later child either.

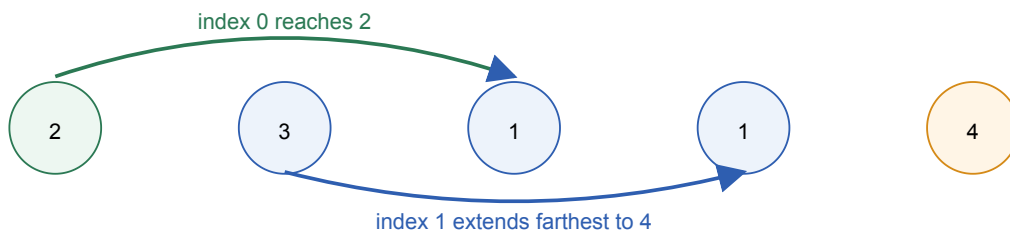
Why use the sorted arrays?

Comparing the original unsorted data destroys the two-pointer guarantee.

05 Jump Game

MUST KNOW

`farthest` is not the current position. It is the farthest index reachable by any route examined so far.



Invariant: before processing index i , every index up to `farthest` is reachable. If $i > \text{farthest}$, no known route can reach the current position.

```
def can_jump(numbers):
    farthest = 0

    for index, jump_length in enumerate(numbers):
        if index > farthest:
            return False
        farthest = max(farthest, index + jump_length)
    return True
```

Why `index > farthest`, not `>=`?

When they are equal, the current index is exactly reachable and must still be processed.

Why does `[0]` return True?

The starting position is already the destination.

06 Minimum Arrows

Each balloon is an interval. Place the current arrow at the earliest end position and reuse it while intervals still overlap that point.

```
def minimum_arrows(intervals):
    if not intervals:
        return 0

    sorted_intervals = sorted(intervals, key=lambda item: item[1])
    arrows = 1
    current_arrow_position = sorted_intervals[0][1]

    for start, end in sorted_intervals[1:]:
        if start > current_arrow_position:
            arrows += 1
            current_arrow_position = end
    return arrows
```

Why the earliest end? It is the leftmost safe point that still hits the current balloon, leaving maximum overlap with later balloons.

Why `start > current_arrow_position`?

Equality still means the arrow lies on the balloon's boundary and hits it.

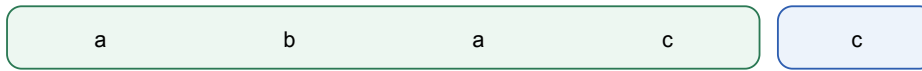
What is wrong with `current_arrow_position = sorted_intervals[1]`?

It returns an entire interval tuple. Use `sorted_intervals[0][1]` to take the first interval's end value.

07 Partition Labels

Record each character's last occurrence. The current segment must extend to the farthest last occurrence of every character it contains.

"abac"



segment length 4

length 1

Cut only when `index == end`: every letter in the segment has finished appearing.

partition_labels · last occurrence + expanding boundary

last_index

```
def partition_labels(text):
    if not text:
        return []

    last_index = {}
    for index, character in enumerate(text):
        last_index[character] = index

    result = []
    start = end = 0
    for index, character in enumerate(text):
        end = max(end, last_index[character])
        if index == end:
            result.append(end - start + 1)
            start = index + 1
    return result
```

Why is the cut safe? At `index == end`, no character already in the segment can reappear later.

Is this grouping consecutive equal letters?

No. It groups letters whose occurrence ranges overlap, even when equal letters are far apart.

08 Best Time to Buy and Sell Stock

MUST KNOW

Maintain the lowest price seen so far and the best profit available so far. Buying must happen before selling.

max_profit · one pass, two states

single pass

```
def max_profit(prices):
    if not prices:
        return 0

    lowest_price = prices[0]
    best_profit = 0

    for current_price in prices[1:]:
        lowest_price = min(lowest_price, current_price)
        possible_profit = current_price - lowest_price
        best_profit = max(best_profit, possible_profit)
    return best_profit
```

State 1

`lowest_price` summarizes the best buying opportunity so far.

State 2

`best_profit` preserves the best completed trade so far.

Why not assign today's profit directly?

Today's candidate can be worse than the historical best.
Always use `max(best_profit, candidate)`.

Why return 0 when prices always fall?

Choosing not to trade is valid and yields profit 0.

09 Bug + FAQ Summary

Four Common Mistakes

Mistake	Why it fails	Correct approach
Writing greedy as backtracking	A greedy proof should eliminate the need to retreat.	Sort or maintain state, then make one pass.
Sorting, then using the original list	The ordering guarantee disappears.	Use the sorted variables consistently.
Treating <code>farthest</code> as a position	It represents a reachable boundary across routes.	Keep “current index” and “coverage boundary” distinct.
Ignoring touching boundaries	<code>></code> and <code>>=</code> encode different rules.	Confirm whether endpoints may touch.

Five High-frequency FAQs

Why is greedy code simpler than backtracking?

It stores no path, explores no search tree, and undoes no choice. The hard part is the proof of safety outside the code.

Why can Coin Change not always use greedy?

Largest-first may not minimize the number of coins. `[1, 3, 4]` for 6 is the standard counterexample.

Why is `[0]` True in Jump Game?

The starting position is already the destination.

Why can Partition Labels not cut immediately?

A character may appear later. Wait until every character in the current segment has reached its last occurrence.

Why shoot at the earliest ending position?

It is the leftmost safe point that preserves the largest possible overlap with later balloons.

Return to the universal test: What state matters? Is the choice safe? Must several routes be explored? A “yes” to the last question usually means backtracking or DP, not greedy.

10 Practice Checklist

Before coding each problem, name the greedy state variable and state why the local choice is safe.

◆ PRACTICE

1. **E** `max_non_overlapping_intervals([(1,2),(2,3),(3,4)])` — expected 3; tests touching boundaries.
2. **M** `greedy_coin_change([1,3,4], 6)` — greedy returns [4,1,1]; explain why it is not optimal.
3. **E** `find_content_children([2,3,4],[1,2,3])` — expected 2; sorting + two pointers.
4. **E** `can_jump([2,0,0])` — expected True; track `farthest`.
5. **M** `minimum_arrows([(1,2),(2,3),(3,4),(4,5)])` — expected 2; boundary hits remain valid.
6. **M** `partition_labels("abac")` — expected [3,1]; use last occurrences.
7. **E** `max_profit([7,6,4,3,1])` — expected 0; no profitable trade.

✓ Chapter Checklist

- ☐ Greedy means the safest local optimum at each step.
- ☐ Do not equate greedy with “always pick max/min.”
- ☐ Decide whether sorting creates the needed order.
- ☐ Name the state variable before coding.
- ☐ Coin Change is not always greedy-optimal.
- ☐ `farthest` is a boundary, not a position.
- ☐ Interval problems often sort by end time.
- ☐ Stock profit requires buy before sell.
- ☐ Distinguish Greedy, DP, and Backtracking.
- ☐ Rebuild each solution from its invariant.

? Fast Decision Guide

Choose Greedy when...

one safe local rule plus a small state is enough, and earlier choices never need revision.

Choose DP when...

several subproblem answers must be compared to guarantee the global optimum.

Choose Backtracking when...

the task requires exploring alternative paths, constraints, or complete configurations.

Final summary: Greedy code is often short because the proof has already removed unnecessary branches. The implementation is usually sorting or one pass; the intellectual work is the invariant and exchange argument.

Core templates

Intervals → sort by end
Matching → sort + two pointers
Reachability → maintain a boundary

Core warnings

Local best needs a proof.
Boundary equality changes correctness.
Never overwrite a better historical state.